Recap: Linear programmes (LP)

An optimization problem that aims to **maximize or minimize a linear objective function**, subject to **linear equality and/or inequality constraints**.

$$\begin{aligned} &\text{Maximize} && \mathbf{c}^T\mathbf{x} \\ &\text{Subject to} && \mathbf{Ax} \leq \mathbf{b} \\ & && \mathbf{x} \geq 0 \end{aligned}$$

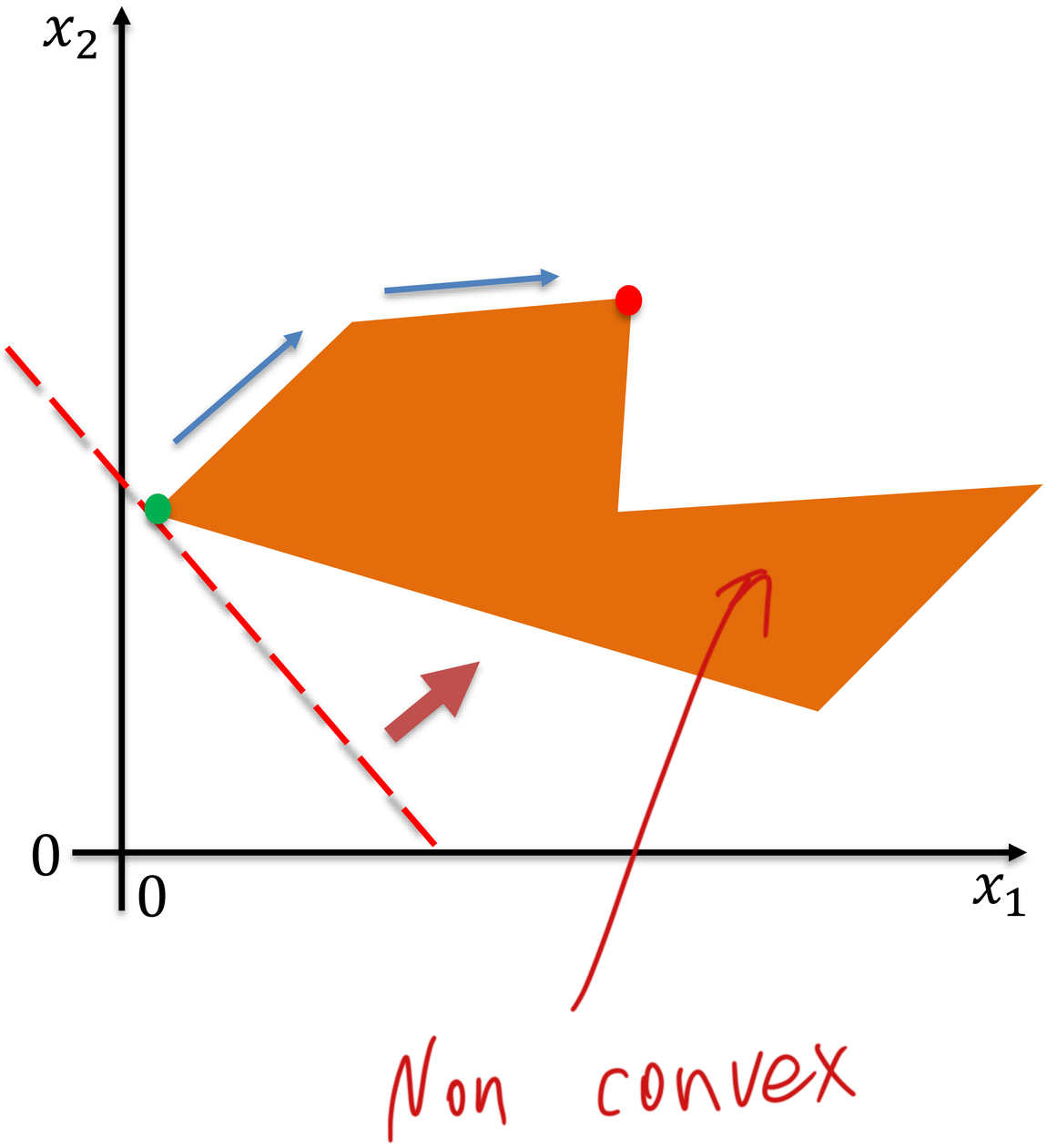
Solving LPs:

Graphical method: applicable when there are only two variables.

Simplex algorithm: efficient for large-scale LPs

Fundamental theorem: Optimal solutions occur on the boundary of the feasible region — typically at its vertices (or along an edge between vertices).

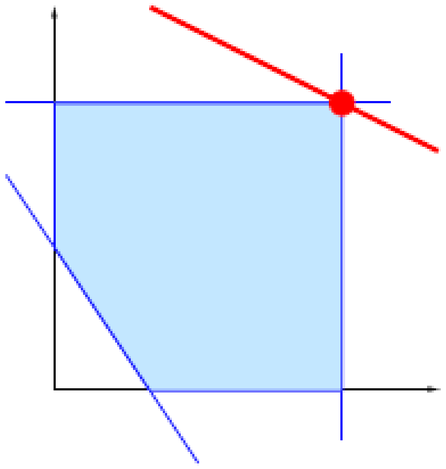
Will Simplex algorithm terminate early?



The Simplex algorithm is specifically designed for linear programs, where the feasible region is guaranteed to be **convex**.

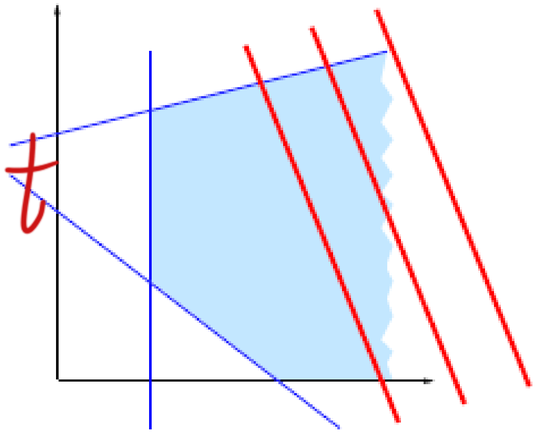
Non-linear constraints can create non-convex regions, where multiple local optima may exist, and the global optimum may occur at interior points rather than at vertices. In such cases, the Simplex algorithm cannot be applied.

Recap: Different cases of LP outcomes

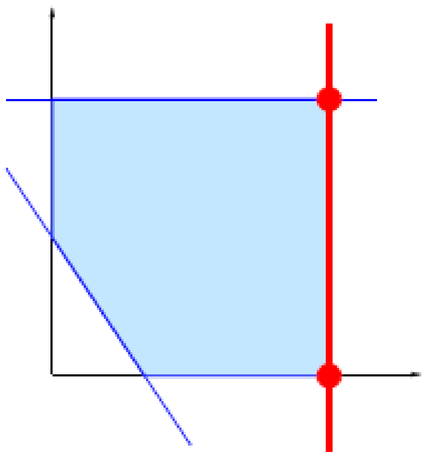


Feasible region may not be bounded

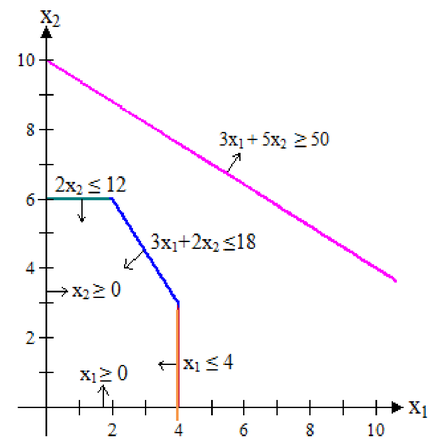
(a) Single optimal solution



(c) No optimal solution



(b) Infinite number of optimal solution

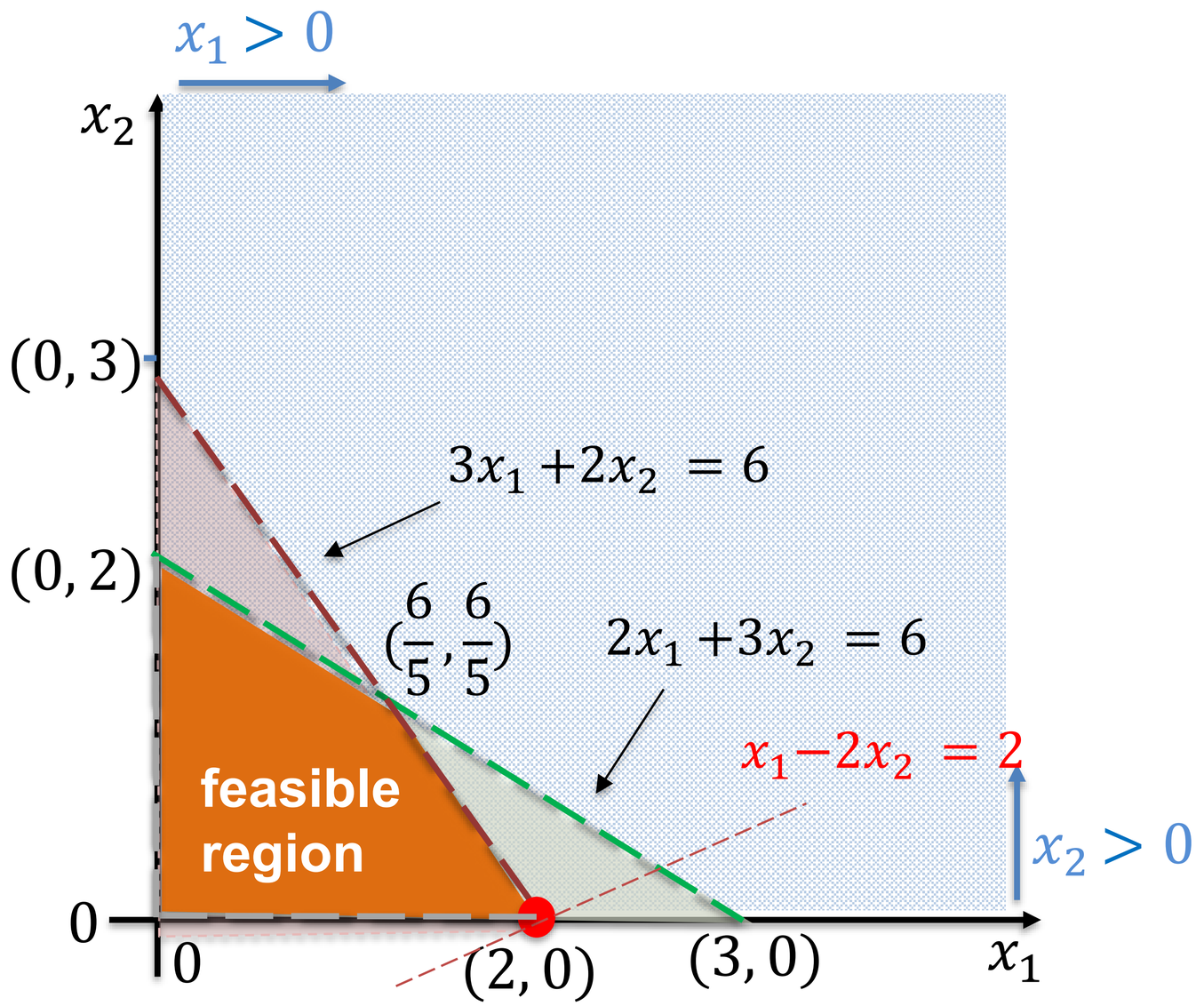


(d) No optimal solution

Why not strict inequality?

Maximize $x_1 - 2x_2$

Subject to:
$x_1 > 0$
$x_2 > 0$
$3x_1 + 2x_2 < 6$
$2x_1 + 3x_2 < 6$



- The feasible set becomes **open** if strict inequalities are used.
- This means the boundary line is excluded from the feasible region.
- But in LP, the optimal solution lies on the boundary.

Linear vs. affine in LP

Strict mathematics

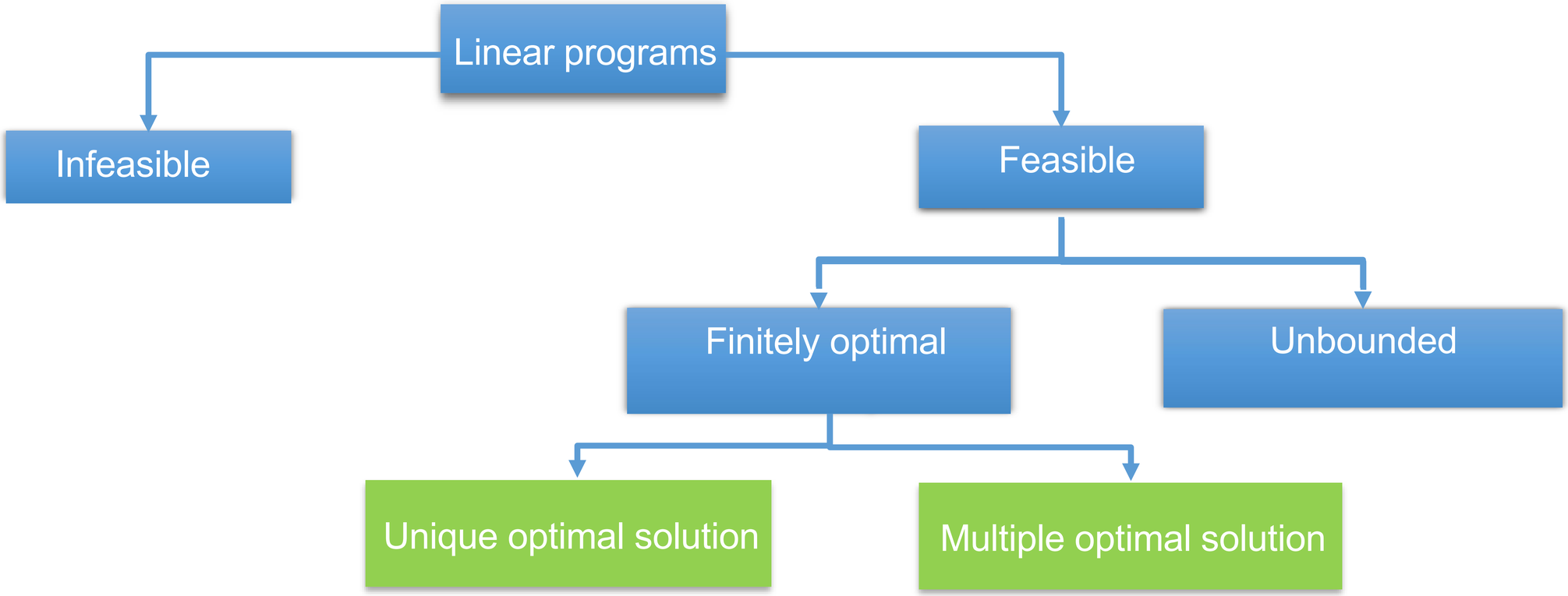
- Linear function: passes through the origin (e.g., $f(x_1, x_2) = 3x_1 - 2x_2$).
- Affine function: linear part + constant term (e.g., $x_1 + x_2 \leq 15$).

Why we still say "linear programming"

- In optimization, "linear" includes affine functions.
- Both constraints and objective functions may have constant terms.
- Constants don't change the feasible region shape or the optimal solution (only shift values).

Takeaway: In LP, *linear* = *linear/affine*.

Recap: Different cases of LP outcomes



Q3 (iii) Solving the problem in Python using a library like CVXPY

Introduction to CVXPY library:

- CVXPY is a Python library* for modeling and solving convex optimization problems, including linear and integer programs.

- It provides a simple and readable way to define decision variables, objective functions, and constraints in a mathematical style.

- It supports a variety of powerful solvers (including simplex-based solvers such as HiGHS) to solve problems under the hood.

*CVXPY is not included in Anaconda by default — you'll need to install it separately.
Reference: https://www.cvxpy.org/api_reference/cvxpy.html

Q3 (iii) Solving the problem in Python using a library like CVXPY (cont.)

```python
# Import libraries
import matplotlib.pyplot as plt # For plotting
import cvxpy as cp  # For solving linear programs
import numpy as np  # For numerical computations (used for plotting here)

# 1. Define variables
x_A = cp.Variable(name="product_A", integer=True)
x_B = cp.Variable(name="product_B", integer=True)

# 2. Define objective
objective = cp.Maximize(40 * x_A + 30 * x_B)

# 3. Define constraints
constraints = [
    x_A >= 0,              # Non-negativity for A
    x_B >= 0,              # Non-negativity for B
    2 * x_A + 1 * x_B <= 100,    # Machine time limit
    x_B >= 10,                   # Minimum product B
    x_A + x_B <= 40              # Production capacity
]

# 4. Creates the CVXPY optimization proble
prob = cp.Problem(objective, constraints)

# 5. Solve the LP problem
prob.solve()

# 6. Results
xA_opt = x_A.value # Get the optimal value of decisi
xB_opt = x_B.value # Get the optimal value of decisi
max_profit = prob.value # Get the maximum profit fro
problem_status = prob.status # Get the solution stat
```

cp.Variable(name="x", integer=True): Define optimization variables.
-name: (string) a label to the variable (optional, for readability)
-integer: (boolean) if True, the variable is restricted to be integers values; otherwise it is continuous by default.

cp.Maximize(): Define objective, tells CVXPY that we want to **maximize** the expression inside

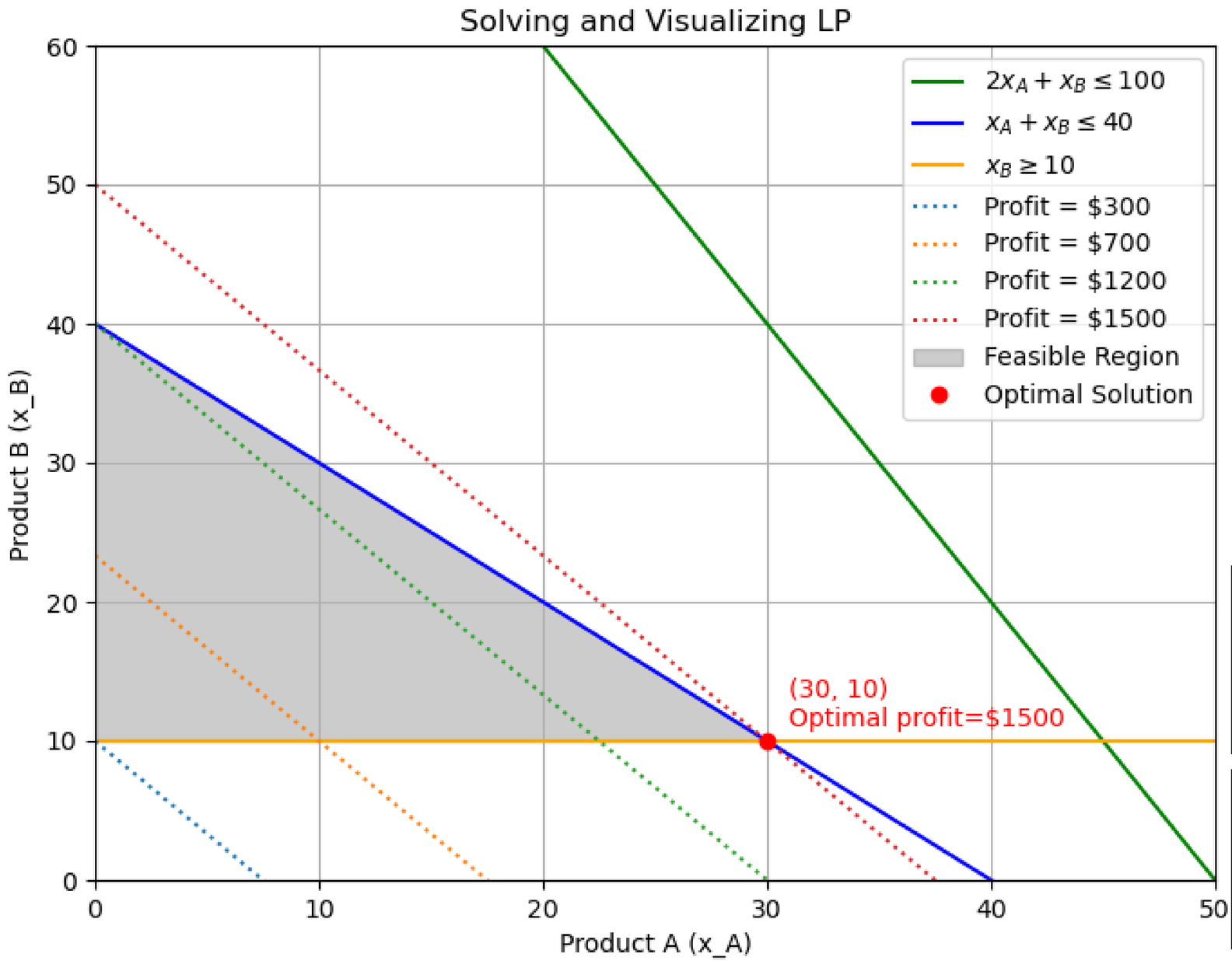
.constraints = []: a list that will hold all the problem's constraints.
-each item in the list is a CVXPY expression representing a constraint.
-internally, CVXPY stores these constraints as **objects** that know the left-hand side (e.g., x_A+x_B), the inequality type (e.g., <=), the right-hand side (e.g., 40)
-when you pass the list to prob.solve(), CVXPY interprets all the objects together as the **feasible region** for the optimization.

cp.Problem(objective, constraints**):** creates a CVXPY **problem object**. It combines the objective and constraints.

.solve(): runs the solver to find the best solution. For linear program, the solver may use the Simplex algorithm.

.value (for variables): gives the values of the decision variables.
.value (for problem): Gives the optimal value of the objective function.
.status: Tells you what happened when solving:
'optimal' → found a solution
'infeasible' → no solution satisfies all constraints
'unbounded' → objective can improve indefinitely

Q3 (iii) Solving the problem in Python using a library like CVXPY (cont.)



Maximize $z = 40x_A + 30x_B$

Subject to: $2x_A + x_B \leq 100$

$x_B \geq 10$

$x_A + x_B \leq 40$

$x_A, x_B \geq 0$

```
# Output results
print("\nSolution Status:", problem_status)
print("Optimal number of units to produce:")
print("Product A:", xA_opt)
print("Product B:", xB_opt)
print("Maximum Profit: $", max_profit)
```

```
Solution Status: optimal
Optimal number of units to produce:
Product A: 30.0
Product B: 10.0
Maximum Profit: $ 1500.0
```

*Visualization code is included in the shared Python file. While it's not required for exams, visualization is a valuable tool in real-world practice---it helps us better interpret results and understand model performance.

Q3 Code highlights – Key Python libraries

```
import matplotlib.pyplot as plt
from cvxpy as cp
import numpy as np
```

•**matplotlib**: A library for creating visualizations such as charts and plots to help interpret data and results.

•**cvxpy**: A Python library for **linear and convex optimization**, allowing us to define and solve optimization problems.

•**numpy**: A fundamental package for numerical computing in Python, offering support for arrays, math functions, and linear algebra.

By convention, we import libraries with short names for convenience: *numpy* as *np, cvxpy* as *cp.*

Q3 Code highlights – CVXPY library

```
from cvxpy as cp

cp.Variable(name=None, integer=False)
```

Defines a decision variable in the optimization problem.

- name: variable identifier (optional, e.g., "x_A").
- integer: if True, variable is restricted to integer values; otherwise it is continuous (default).

Example:

x = cp.Variable(name=“product x", integer=True)
creates an integer variable x

```python
# 1. Define variables
x_A = cp.Variable(name="product_A", integer=True)     # integer variable
x_B = cp.Variable(name="product_B", integer=True)     # integer variable
```

Reference: CVXPY-tutorial-TUT4.ipynb

Reference: https://www.cvxpy.org/api_reference/cvxpy.html

Q3 Code highlights – CVXPY library (cont.)

`cp.Maximize() / cp.Minimize()`

Specifies the objective function of the optimization problem.
cp.Maximize(expr) → maximize the expression expr.
cp.Minimize(expr) → minimize the expression expr.
For example: objective = cp.Maximize(40 * x_A + 30 * x_B)

`constraints = []`

Defines a list to hold all the problem's constraints.
Each item in the list is a CVXPY expression representing a rule for the variables. Start empty, then add constraints like inequalities or equalities. For example:

```
constraints = [
    x >= 0,          # Non-negativity
    x + y <= 10      # Total limit
]
```

```python
# 2. Define objective
objective = cp.Maximize(40 * x_A + 30 * x_B)

# 3. Define constraints
constraints = [
    x_A >= 0,                # Non-negativity for A
    x_B >= 0,                # Non-negativity for B
    2 * x_A + 1 * x_B <= 100,   # Machine time limit
    x_B >= 10,                  # Minimum product B
    x_A + x_B <= 40             # Production capacity
]
```

Q3 Code highlights – CVXPY library (cont.)

`cp.Problem(objective, constraints)`
Combines the objective and constraints into a CVXPY problem object.
This object represents the entire optimization problem.
For example:
`prob = cp.Problem(objective, constraints)`

`prob.solve()`
Tells CVXPY to actually solve the optimization problem.
CVXPY passes the problem to an underlying solver (like HiGHS, ECOS, or others). *For linear programs, the solver may use the Simplex algorithm.*
After solving, we get:
Optimal variable values: x.value, y.value, …
Optimal objective value: prob.value
Solution status: prob.status (optimal, infeasable, unbounded)

```
# 4. Creates the CVXPY optimization problem by combining the objective and the constraints
prob = cp.Problem(objective, constraints)

# 5. Solve the LP problem
prob.solve()
```

Reference: You may refer to simplex_algorithm.py for simplex algorithm implementation

Q3 Code highlights – CVXPY library (cont.)

Get results:

.value (for variables)
Gives the values of the decision variables that produce the optimal objective value.

.value (for problem)
Gives the optimal value of the objective function.

.status
Tells you what happened when solving:
'*optimal*' : found a solution
'*infeasible*': no solution satisfies all constraints
'*unbounded*' : objective can improve indefinitely

```
# 6. Results
xA_opt = x_A.value # Get the optimal value of decision variable x_A
xB_opt = x_B.value # Get the optimal value of decision variable x_B
max_profit = prob.value # Get the maximum profit from the objective function
problem_status = prob.status # Get the solution status
```

In CVXPY (and most LP solvers), .status only tells you whether a solution was found. It does not reveal whether the solution is *unique* or if *multiple* optimal solutions exist.

Integer Linear Programming (brief introduction)

Maximize y

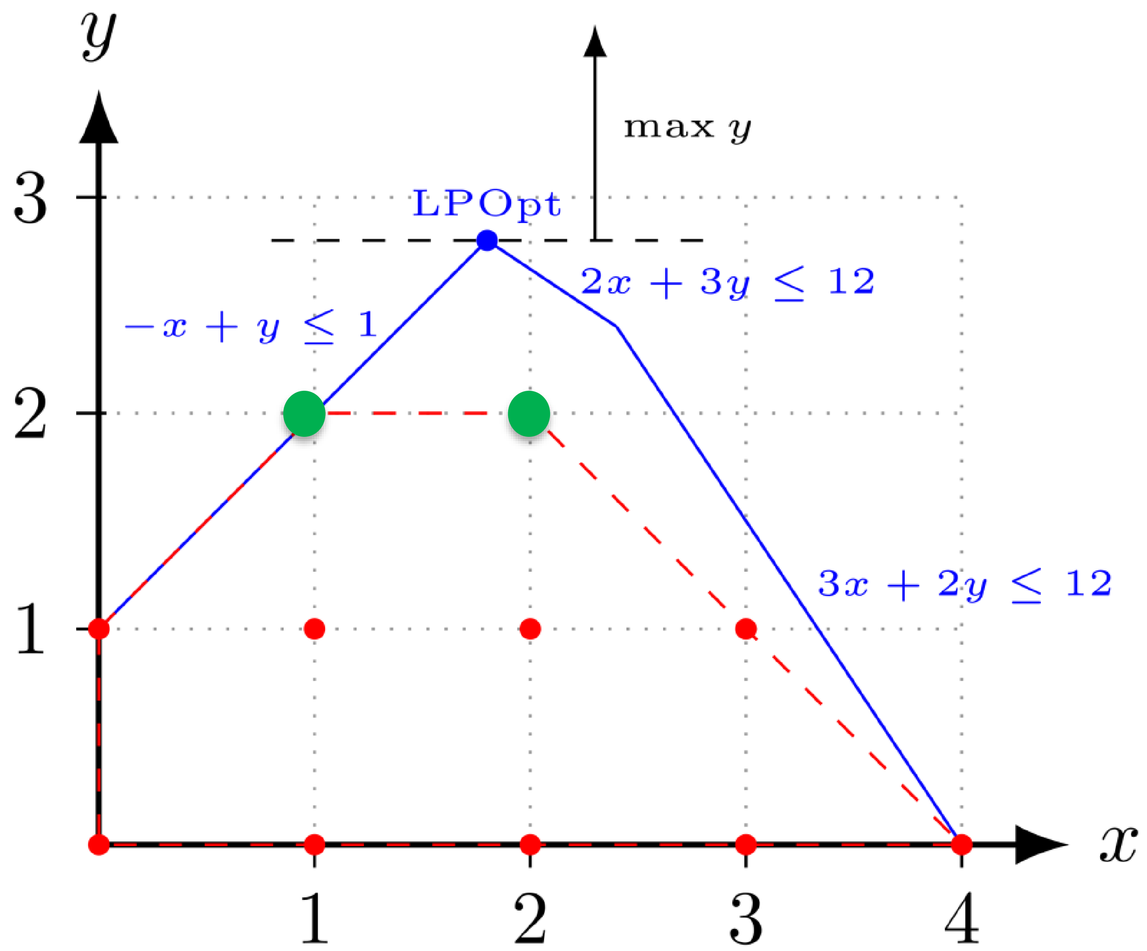
Subject to $-x + y \leq 1$

$3x + 2y \leq 12$

$2x + 3y \leq 12$

$x, y \geq 0$

$x, y \in \mathbb{Z}$



Challenge: The optimal integer linear programming (ILP) solution must be an integer point, which can lie **inside** the LP's feasible region, not just on its boundary.

Solution Strategy: We first solve the easier **LP Relaxation** (ignoring integer constraints) to get a bound, then use algorithms like branch and bound to find the best integer solution.

Reference: https://web.mit.edu/15.053/www/AMP-Chapter-09.pdf

LP vs ILP

Feature	Linear Programming (LP)	Integer Linear Programming (ILP)
Decision Variables	Continuous (any real number)	Integer (whole numbers only)
Feasible Region	A continuous region (includes all points inside and on the boundaries)	Discrete set of points inside the feasible region or on the boundary
Optimal Solution Location	At a vertex (or along an edge) of the feasible region	Can be at a vertex (on the edge) or inside the region
Solution method	Relatively easy — efficient algorithms (like Simplex, Interior Point)	Harder — often requires special methods (Branch & Bound, Cutting Planes)
Practical Examples	Resource allocation with divisible quantities (e.g., fractional production, mixing fuels, maximum flow, diet problem, transportation problem)	Situations requiring whole numbers (e.g., number of buses, yes/no decisions, stock trading with indivisible units)

Q3 (iii) Method 2: Solving the problem in Python using a library like PuLP

Introduction to PuLP library:

- PuLP is a Python library* for modeling and solving linear and integer programming problems.

- It offers a simple, readable way to define decision variables, objective functions, and constraints.

- It solves linear programs using powerful solvers, such as the Simplex-based CBC.

*PuLP is not included in Anaconda by default — you'll need to install it separately.
Reference: https://coin-or.github.io/pulp/guides/index.html

Q3 (iii) Method 2: Solving the problem in Python using a library like PuLP

```python
# Import libraries
import matplotlib.pyplot as plt # For plotting
import numpy as np  # For numerical computations (used for plotting here)
from pulp import *  # PuLP: Python library for modeling linear programming problems

# Define the LP problem
prob = LpProblem("Maximize_Profit", LpMaximize)

# Decision variables
x_A = LpVariable("x_A", lowBound=0)
x_B = LpVariable("x_B", lowBound=0)

# Objective function: Maximize profit
prob += 40 * x_A + 30 * x_B         # Total profit

# Constraints
prob += 2 * x_A + 1 * x_B <= 100    # Machine time
prob += x_B >= 10                   # Minimum product B
prob += x_A + x_B <= 40             # Production capacity

# Solve the LP problem
prob.solve()

# Results
xA_opt = value(x_A) # Get the optimal value of decision variable x_A
xB_opt = value(x_B) # Get the optimal value of decision variable x_B
max_profit = value(prob.objective) # Get the maximum profit from the objective function
```

LpProblem(name, sense): creates a new LP problem.
- name: just a label (optional, for readability)
- sense: *LpMaximize* (to maximize) or *LpMinimize* (to minimize)

LpVariable(name, lowBound, upBound, cat): defines a variable for LP
- name: variable name (string)
- lowBound: lower limit (e.g., 0 for non-negativity)
- cat: category of variable (e.g., Continuous, Integer). Default is Continuous.

"**+=**" is used in PuLP to add an expression to the LP model
Here, we are adding the objective function and constraints to 'prob'

.solve(): runs the built-in solver to find the optimal solution (usually using the Simplex algorithm)

.value(variable or expression):
Extracts the numerical value from a *solved* variable or objective function.

Q3 Code highlights – PuLP library

```
prob = LpProblem("Maximize_Profit", LpMaximize)
```

Creates a new LP problem named "Maximize_Profit" with the goal to maximize the objective function, and stores it in the variable 'prob'.

```
LpProblem(name, sense)
```

Creates a new linear programming problem.

- `name:` a label for the problem (for reference).
- `sense:` specifies whether to maximize (`LpMaximize`) or minimize (`LpMinimize`) the objective.

Reference: Pulp-tutorial-TUT4.ipynb

For more information, check the PuLP documentation: https://coin-or.github.io/pulp/main/includeme.html

Q3 Code highlights – PuLP library (cont.)

```
x_A = LpVariable("x_A", lowBound=0)
x_B = LpVariable("x_B", lowBound=0)
```

Creates two decision variables named "X_A" and "X_B" for the LP problem, with constraint $X\_A, X\_B \geq 0$ (non-negativity).

```
LpVariable(name, lowBound=None, upBound=None, cat='Continuous')
```

Defines a decision variable in the model.

- `name`: variable identifier (e.g., "x_A").
- `lowBound`: minimum value (e.g., 0 for non-negative).
- `upBound`: maximum value (optional).
- `cat`: variable type – `'Continuous'` (default), `'Integer'`, or `'Binary'`.

Example:

```
x = LpVariable("x", lowBound=0, upBound=10, cat='Integer')
```

creates an integer variable x with $0 \leq x \leq 10$.

Q3 Code highlights – PuLP library (cont.)

```
prob += 40 * x_A + 30 * x_B
Add the objective function (to maximize) to the LP problem.

prob += 2 * x_A + 1 * x_B <= 100
prob += x_B >= 10
prob += x_A + x_B <= 40
Add constraints to the LP problem.
```

Context	Meaning of `+=`
Standard Python	Adds and updates the value of a variable. Example: `x += 1` means `x = x + 1`.
PuLP	Adds an expression (objective or constraint) to the LP problem model. Example: `prob += 40 * x_A + 30 * x_B` adds the objective function to the LP.

Q3 Code highlights – PuLP library (cont.)

`prob.solve()`
Solve the LP problem.

`.solve()`
- Runs the solver to find the optimal solution to the defined problem.
- By default, PuLP uses CBC (Coin-or Branch and Cut) as the solver, which is included with PuLP.
- CBC typically applies the *Simplex algorithm* for linear programs.
- This step performs all the necessary computations to maximize or minimize the objective while satisfying the constraints.

Q3 Code highlights – PuLP library (cont.)

```
xA_opt = value(x_A)
xB_opt = value(x_B)
```
Get the optimal values of decision variables x_A, X_B and store them in xA_opt and xB_opt.

```
max_profit = value(prob.objective)
```
Get the maximum profit from the objective function (i.e., the maximum profit).

`value(variable or expression)`

- Extracts the numerical value from a *solved* variable or objective function.
- Used to get the *final results*, such as optimal quantities or maximum profit.

Q3 Code highlights – PuLP library (cont.)

`.status`

The `.status` attribute of a PuLP problem tells you the result of the solver, that is, what happened when it tried to solve your linear program.

PuLP represents status codes as constants in pulp.constants, such as:

- LpStatusOptimal
- LpStatusUnbounded
- LpStatusInfeasible

prob.status (numeric)	LpStatus[prob.status]	Meaning	Case	Action needed
1	Optimal	Solution found that meets all constraints	Feasible and finitely optimal	Use the result
-2	Unbounded	Objective can go to $+\infty$ or $-\infty$	Feasible but unbounded	Check for missing constraints
-1	Infeasible	No solution satisfies all constraints	Infeasible	Re-express or relax constraints
0	Other	Solver failed or returned unexpected result	--	Debug model or solver setup
-3	Undefined	Solver failed or returned unexpected result	--	Debug model or solver setup

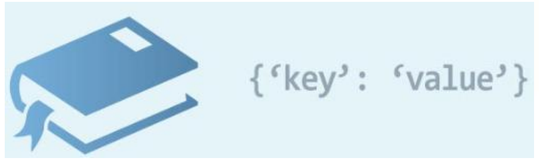
prob is the variable that represents the entire linear programming model in **our** code

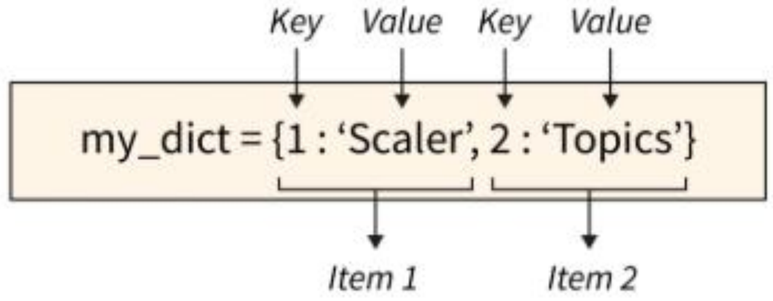
In PuLP (and most LP solvers), .status only indicates whether an optimal solution was found, it does not reveal whether the solution is unique or if multiple optimal solutions exist. That is, prob.status == LpStatusOptimal simply confirms that an optimal solution was found, without specifying how many such solutions there may be.

Q3 Code highlights – PuLP library (cont.)

Item	Type	Description
.status	Integer	gives a number that tells what happened when the solver tried to solve your problem. (e.g., 0, 1, -1, -2, or -3)
LpStatus	Dictionary	PuLP's built-in Python dictionary that maps status codes to their meanings
LpStatus[status]	String	turns that number into a readable message like "Optimal" or "Infeasible".

Python dictionary:






```
If my_dict = {1: 'Topics', 2: 'Join', 'key3': 'Scaler'}:

Then my_dict[2] == 'Join'

And my_dict['key3'] == 'Scaler'
```

```python
LpStatus = {
    -3: "Undefined",      # Solver could not determine the problem status
    -2: "Unbounded",      # Objective value can increase without limit
    -1: "Infeasible",     # No solution satisfies all constraints
     0: "Not Solved",     # Problem was defined but not solved yet
     1: "Optimal"         # Solver found the best solution under given constraints
}
```

Reference: https://www.w3schools.com/python/python_dictionaries.asp

Q3 PuLP vs CVXPY library

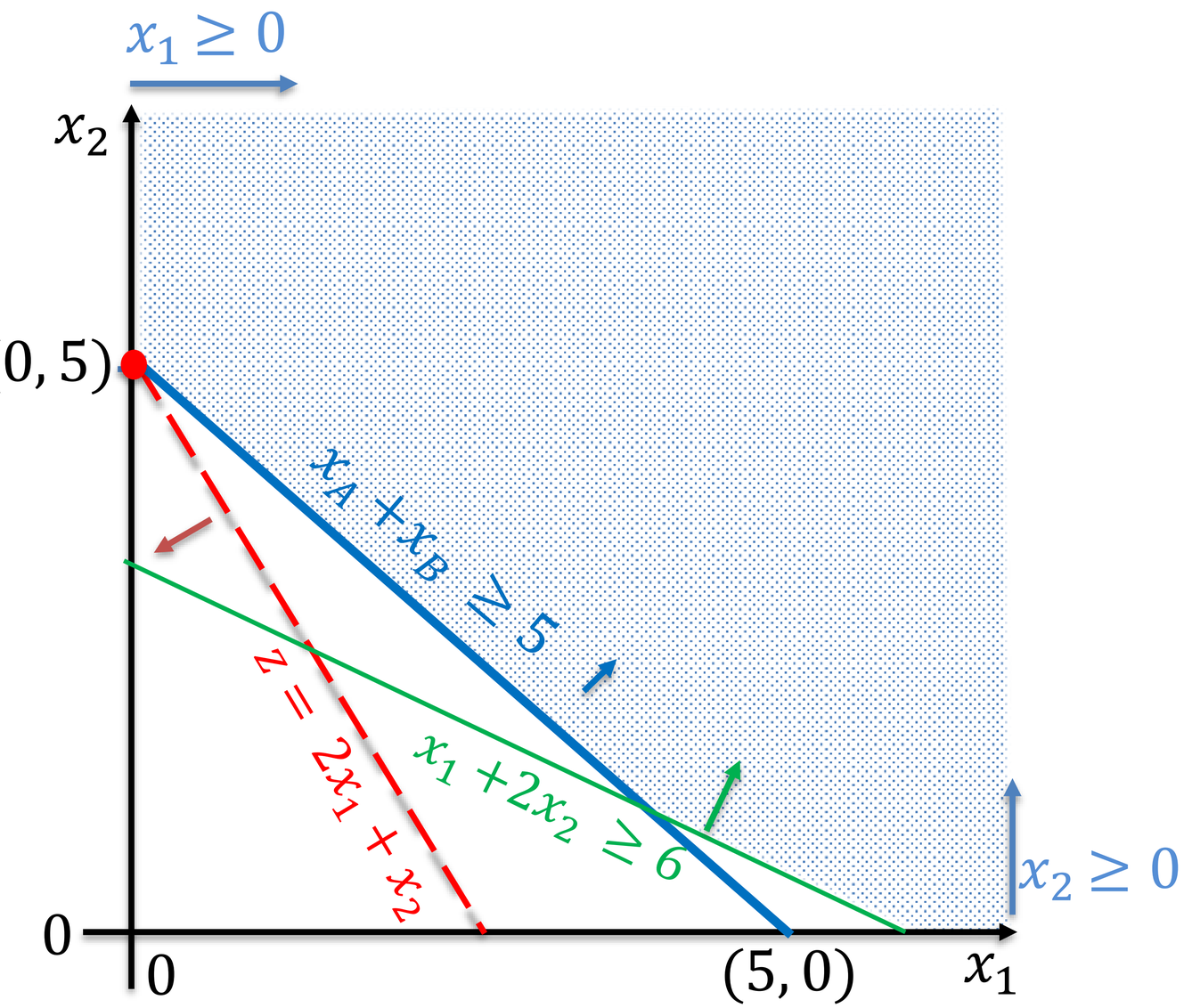
Feature	PuLP	CVXPY
Problem Focus	Linear and integer LP/ILP	General convex optimization (LP, QP, SDP, etc.)
Integer/Binary Variables	Directly supported	Supported, but sometimes may need specific solvers
Syntax Style	Problem-focused: LpProblem, LpVariable, += for objective and constraints	Mathematical-style: cp.Problem(cp.Maximize(...), [constraints])
Solver Integration	Built-in (CBC), GLPK, CPLEX, Gurobi	Many solvers; sometimes requires explicit selection

Q4 The given linear program is:

Minimize $z = 2x_1 + x_2$

subject to:

$$x_1 + x_2 \geq 5$$
$$x_1 + 2x_2 \geq 6$$
$$x_1, x_2 \geq 0$$



Method 1: Graphical method

Ans: Bounded and feasible

$x_1 = 0,\ x_2 = 5,\qquad z = 5$

The LP is bounded, although the feasible region is unbounded.

Resources: https://www.desmos.com/calculator

Q4 Method 2: Using Python

```python
import cvxpy as cp  # For solving linear programs

# 1. Define variables
x_1 = cp.Variable()  # defines a continuous decision variable x_1.
x_2 = cp.Variable()  # defines a continuous decision variable x_2.

# 2. Define objective
objective = cp.Minimize(2 * x_1 + x_2)

# 3. Define constraints
constraints = [
    x_1 >= 0,
    x_2 >= 0,
    x_1 + x_2 >= 5,
    x_1 + 2 * x_2 >= 6
]

# 4. Create a CVXPY problem object
prob = cp.Problem(objective, constraints)

# 5. Solve the LP problem
prob.solve()
```

Minimize $z = 2x_1 + x_2$

subject to:

$$x_1 + x_2 \geq 5$$
$$x_1 + 2x_2 \geq 6$$
$$x_1, x_2 \geq 0$$

```
Solution Status: optimal
x1: -9.328082293135338e-11
x2: 5.000000000322774
Minimum objective value: 5.000000000136212
```

*The tiny differences occur because CVXPY uses numerical solvers with floating-point arithmetic, so it returns very small approximations close to the exact graphical solution rather than exact integers like 0 or 5.

Q4' The given linear program is:

Maximize $z = 2x_1 + x_2$

subject to:

$$x_1 + x_2 \geq 5$$
$$x_1, x_2 \geq 0$$

- Bounded and feasible
- Feasible but not bounded ✓
- Bounded but not feasible
- Neither bounded nor feasible

Q5 Formulating the problem

Maximize benefit by selecting the best course combination within limits.

- Choose courses within a maximum credit limit (20 credits)
- Stay within a weekly workload limit (40 hours)
- Avoid time slot conflicts
- Can select no more than 3 courses due to personal time and cognitive limits.
- Select only whole courses (integer decision)

Decision variables:

$x_i = 1$ if course i is selected, 0 otherwise, $i = 1, \dots, 5$

Objective Function (maximize academic benefit):

Maximize $Z = 9x_1 + 7x_2 + 8x_3 + 5x_4 + 6x_5$

subject to:

- *Total credits*: $4x_1 + 4x_2 + 4x_3 + 2x_4 + 3x_5 \leq 20$
- *Total weekly workload*: $10x_1 + 9x_2 + 11x_3 + 5x_4 + 7x_5 \leq 40$
- *No time conflicts*: $x_1 + x_2 \leq 1$ (EE2213 and MA2102 conflict (both Mon 10-12))
- *Total no. of courses:* $x_1 + x_2 + x_3 + x_4 + x_5 \leq 3$
- *Binary decision:* $x_i \in \{0, 1\} \quad i = 1, \dots, 5$

ID	Course	Credits	Workload per week	Time Slot	Benefit score
1	EE2213	4	10 hrs	Mon 10–12	9
2	MA2101	4	9 hrs	Mon 10–12	7
3	CDE2212	4	11 hrs	Tue 2–5	8
4	CE3201	2	5 hrs	Wed 3–5	5
5	EE3801	3	7 hrs	Thu 1–3	6

Q5 Method 1: Brute-force method

There are only 5 binary decision variables (x_1 to x_5), each taking values from {0, 1}.

That gives a total of $2^5 = 32$ possible combinations of course selections.

- Checks whether each combination satisfies all the constraints
- Evaluates the objective function (total benefit) for valid combinations.
- Returns the one with the highest academic benefit.

When the number of variables becomes large, for example, 20 courses:

20 binary decisions → 2^{20} = 1,048,576 combinations

That becomes computationally expensive.

Q5 Method 2: Using CVXPY library

```python
# Import libraries
import cvxpy as cp

# Decision variables: 1 if course is selected, 0 otherwise
x1 = cp.Variable(boolean=True, name='EE2213')
x2 = cp.Variable(boolean=True, name='MA2101')
x3 = cp.Variable(boolean=True, name='CDE2212')
x4 = cp.Variable(boolean=True, name='CE3201')
x5 = cp.Variable(boolean=True, name='EE3801')

# Objective function: maximize total benefit
objective = cp.Maximize(9*x1 + 7*x2 + 8*x3 + 5*x4 + 6*x5)

# Constraints
constraints = [
    4*x1 + 4*x2 + 4*x3 + 2*x4 + 3*x5 <= 20,   # Credit limit
    10*x1 + 9*x2 + 11*x3 + 5*x4 + 7*x5 <= 40, # Workload limit
    x1 + x2 <= 1,                             # Time conflict
    x1 + x2 + x3 + x4 + x5 <= 3               # Limit on total number of courses
]

# Define the problem
prob = cp.Problem(objective, constraints)

# Solve the problem
prob.solve()
```

cp.Variable(boolean=True, name="x"): Define optimization variables.
-boolean=True: the variable can only take values {0, 1}; often used when our decision is yes/no choice.
-name: (string) a label to the variable (optional, for readability)

cp.Maximize(): Define objective, tells CVXPY that we want to **maximize** the expression inside

.constraints = []: a list that will hold all the problem's constraints.
-each item in the list is a CVXPY expression representing a constraint.

cp.Problem(objective, constraints**):** creates a CVXPY problem object.

.solve(): runs the solver to find the best solution.

Q5 Method 2: Using CVXPY library (cont.)

```python
# Output results
status_text = prob.status

print("Solver Status:", status_text)

# Show selected courses if solution is optimal
if status_text == "optimal":
    print("Selected courses:")
    for var in [x1, x2, x3, x4, x5]:
        if var.value == 1:
            print(var.name(), "is selected")
    print("Total benefit:", prob.value)
elif status_text == "unbounded":
    print("The problem is unbounded. Add more constraints.")
elif status_text == "infeasible":
    print("The problem has no feasible solution.")
else:
    print("Solver returned an undefined status. Please check your model.")
```

.value (for variables): gives the values of the decision variables.
.value (for problem): Gives the optimal value of the objective function.
.status: Tells you what happened when solving:
'optimal' → found a solution
'infeasible' → no solution satisfies all constraints
'unbounded' → objective can improve indefinitely

Q5 Method 3: Using PuLP library

```python
# Import libraries
from pulp import *

# Define the problem
prob = LpProblem("Course_Selection", LpMaximize)

# Decision variables: 1 if course is selected, 0 otherwise
x1 = LpVariable('EE2213', cat='Binary')
x2 = LpVariable('MA2101', cat='Binary')
x3 = LpVariable('CDE2212', cat='Binary')
x4 = LpVariable('CE3201', cat='Binary')
x5 = LpVariable('EE3801', cat='Binary')

# Objective function: maximize total benefit
prob += 9*x1 + 7*x2 + 8*x3 + 5*x4 + 6*x5 # Total benefit

# Constraints
prob += 4*x1 + 4*x2 + 4*x3 + 2*x4 + 3*x5 <= 20   # Credit_Limit"
prob += 10*x1 + 9*x2 + 11*x3 + 5*x4 + 7*x5 <= 40 # Workload_Limit
prob += x1 + x2 <= 1 # Time conflict: EE2213 and MA2101 have the same time slot
prob += x1 + x2 + x3 + x4 + x5 <= 3 # Limit on total number of courses

# Solve it
prob.solve()
```

LpProblem(name, sense): creates a new LP problem.
- name: just a label (optional, for readability)
- sense: *LpMaximize* (to maximize) or *LpMinimize* (to minimize)

LpVariable(name, lowBound, upBound, cat): defines a variable for LP
- name: variable name (string)
- lowBound: lower limit (e.g., 0 for non-negativity)
- cat: category of variable (e.g., Continuous, Integer). Default is Continuous.

"**+=**" is used in PuLP to add an expression to the LP model
Here, we are adding the objective function and constraints to 'prob'

.solve(): runs the built-in solver to find the optimal solution (usually using the Simplex algorithm)

Q5 Method 3: Using PuLP library (cont.)

```python
# Output results

# Check and print solver status
status_code = prob.status                # Numeric code (e.g., 1 for Optimal)
status_text = LpStatus[status_code]      # Convert to readable text (e.g., 'Optimal')

print("Solver Status:", status_text)     # Display the outcome

# Show selected courses if solution is optimal
if status_text == "Optimal":
    print("Selected courses:")
    for var in prob.variables():
        if var.varValue == 1:
            print(var.name, "is selected")
    print("Total benefit:", value(prob.objective))
elif status_text == "Unbounded":
    print("The problem is unbounded. Add more constraints.")
elif status_text == "Infeasible":
    print("The problem has no feasible solution.")
else:
    print("Solver returned an undefined status. Please check your model.")
```

.status: returns a numeric code indicating solver's outcome (.e.g., 0, 1, -1, -2, -3)

LpStatus[status_code]: turns that number into a readable message like "Optimal" or "Infeasible".

LpStatus is a built-in Python dictionary that maps status codes to their meanings:

```
{
  0: 'Not Solved',
  1: 'Optimal',
 -1: 'Infeasible',
 -2: 'Unbounded',
 -3: 'Undefined'
}
```

```
Solver Status: Optimal
Selected courses:
CDE2212 is selected
EE2213 is selected
EE3801 is selected
Total benefit: 23.0
```